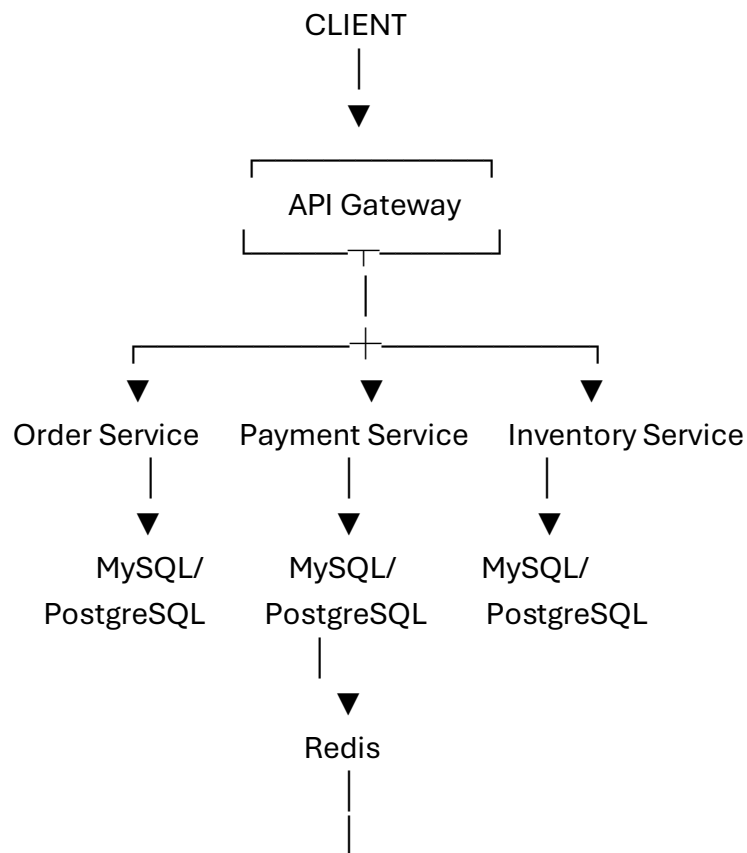


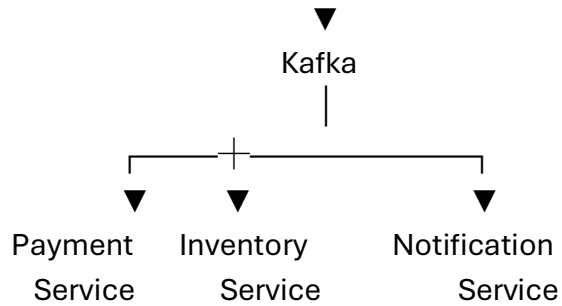
EPIC 06 — Task 05: Distributed Data Architecture

- PostgreSQL/MySQL
- Redis
- Kafka
- Microservices
- Saga
- Document consistency and failure scenarios

We can use the same **Order, Payment, Inventory, and Notification** microservices.

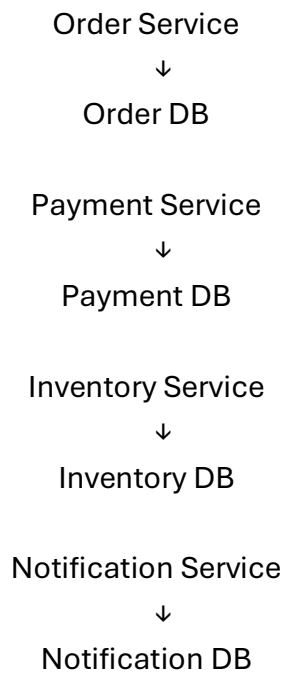
Step 1 — Overall Architecture





Step 2 — Database Ownership

Each microservice owns its own database.



For example:

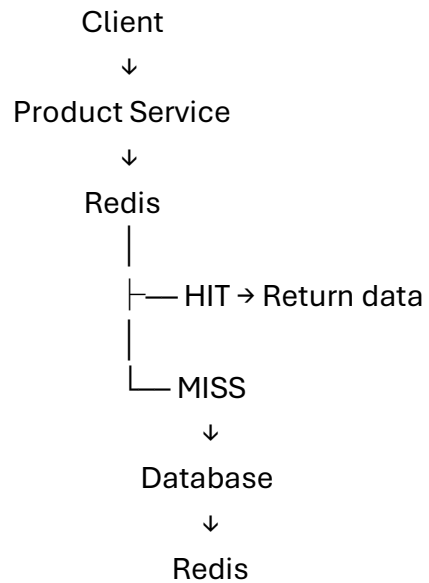
Service	Database	Data
Order Service	order_db	Orders, order items
Payment Service	payment_db	Payments
Inventory Service	inventory_db	Products/stock
Notification Service	notification_db	Notifications

Do not directly share tables between services.

Step 3 — Redis

Redis is used as a **cache** for frequently accessed data.

For example:



Example:

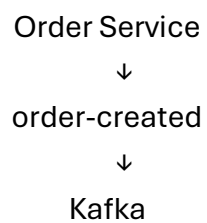
products:101 → Product information

The database remains the source of truth, while Redis improves read performance.

Step 4 — Kafka

Kafka is used for **asynchronous communication** between services.

Example:



↓
Payment Service

After payment:

Payment Service
↓
payment-completed
↓
Kafka
↓
Inventory Service

After inventory:

Inventory Service
↓
inventory-reserved
↓
Kafka
↓
Notification Service

Step 5 — Saga Workflow

The complete order workflow is:

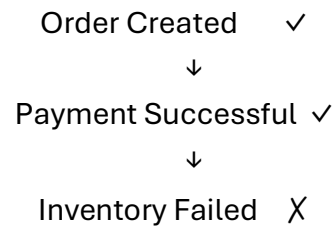
Order
↓
Payment
↓
Inventory
↓
Confirmation

Successful scenario

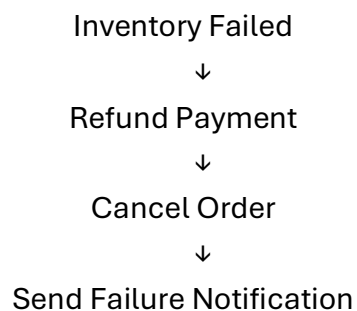


Step 6 — Failure Scenario

Suppose payment succeeds, but inventory fails.



The Saga performs compensation:



Final state:

Order → CANCELLED
Payment → REFUNDED
Inventory → NOT RESERVED

Step 7 — Consistency

Because every service has its own database, the system uses **eventual consistency** rather than one global database transaction.

Example:

Order DB
↓
Order = CREATED

Payment DB
↓
Payment = SUCCESS

Inventory DB
↓
Inventory = RESERVED

Eventually:

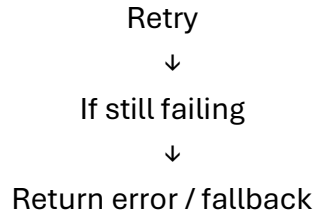
Order = CONFIRMED

Kafka events allow each service to update its own data.

Step 8 — Handling Failures

Database failure

Service
↓
Database unavailable
↓

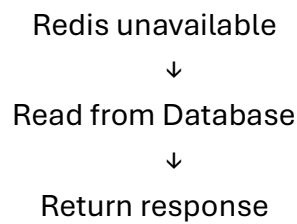


Kafka failure

Use retries mechanisms and durable Kafka messages so events can be processed when the consumer becomes available.

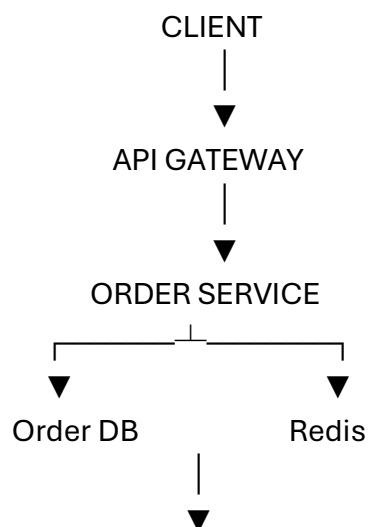
Redis failure

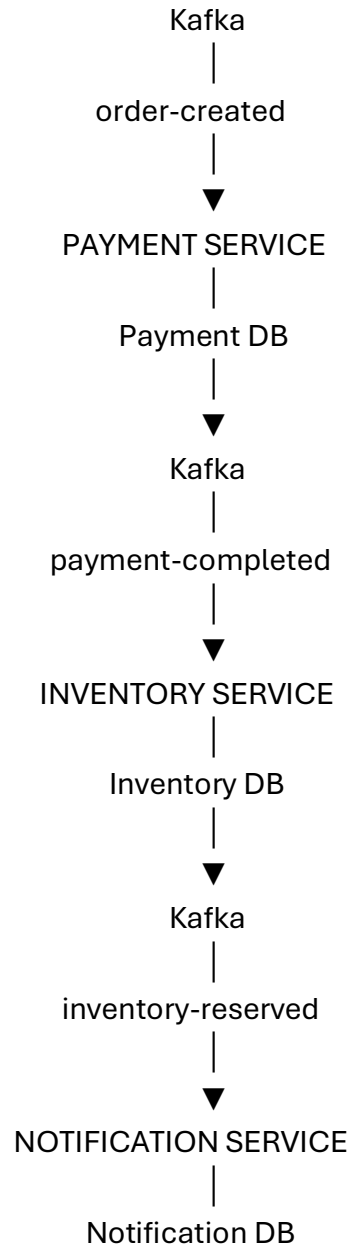
Redis should **not** be the source of truth.



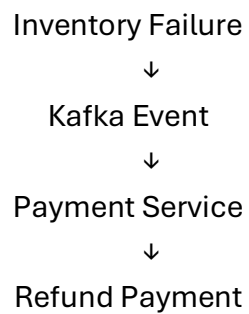
This prevents a cache failure from bringing down the complete application.

Step 9 — Final Architecture





Failure/compensation path





Requirement
PostgreSQL/MySQL
Redis
Kafka
Microservices
Saga
Consistency
Failure handling
Data ownership

Implementation
Separate DB for each service
Caching frequently accessed data
Event-driven communication
Order, Payment, Inventory, Notification
Coordinates distributed transaction
Eventual consistency
Retry, compensation, fallback
Each service owns its database

Task 05 is complete.